Column Generation & Branch-and-Price

A 4-hour practical workshop

Modeling – Pricing – Integrality – Optimality

Antoine Legrain

May 20, 2026

Polytechnique Montréal

What you will leave the room with

By the end of these four hours you should be able to

- Recognise problems that admit an extended formulation with exponentially many variables (routes, patterns, schedules, ...).
- Build the master/pricing pair and run a column-generation loop.
- Write the pricing problem as a Resource-Constrained Shortest Path (RCSPP) and call a labelling algorithm.
- Apply the standard efficiency tricks: multiple columns, heuristic pricing, stabilization, tailing-off.
- Recover an integer solution either heuristically (RMP-MIP, diving) or exactly with branch-and-price, including the Lagrangian bound.
- Implement and debug the loop on a VRPTW instance with HiGHS + the `rcspp` package.

4-hour outline

Time	Topic	Block
00:00–00:30	Why column generation?	Part 1 – Motivation
00:30–01:15	The CG mechanics, RMP & pricing	Part 2 – Mechanics
01:15–02:00	The pricing subproblem (RCSPP)	Part 3 – RCSPP
	15-minute break	
02:15–02:45	Making CG fast	Part 4 – Implementation
02:45–03:15	From LP to integer (heuristics)	Part 5 – IP heuristics
03:15–03:50	Branch-and-price for optimality	Part 6 – B&P
03:50–04:00	VRPTW illustration / lab kickoff	Part 7 – Code

- Hands-on coding alternates with the slides.

Scope and non-scope

Inside the scope

- Modelling with a set-partitioning / set-covering master.
- Pricing as a shortest path with resources.
- Stabilisation, heuristic pricing, multi-column.
- Diving & restricted master heuristic.
- Branching rules that do not kill the pricer.
- The Lagrangian bound for early termination.

Outside the scope

- Full Dantzig–Wolfe / Lagrangian *theory* (we keep an executive summary at the end of Part 2).
- Cut-and-price (a.k.a. branch-cut-and-price) – mentioned, not derived.
- Parallel B&P.
- Stochastic or robust extensions.

Reference for the curious

J. Desrosiers, M. Lübbecke, G. Desaulniers & J.B. Gauthier, *Branch-and-Price*, Springer, 2026 (ISBN 978-3-031-96917-1, open access, the PDF in your folder) covers everything we skip – in particular cut-and-price, stabilisation theory and computational case studies.

Part 1 – Why column generation?

The problem we want to solve

Generic combinatorial optimization problem

$$\min\ c^\top x \quad \text{s.t.} \quad Ax \geq b, \quad x \in X$$

where X encodes combinatorial structure (knapsack, paths, schedules...).

Two ways to write it as a (mixed-)integer linear program.

Compact (arc-flow) formulation

- Variables = "physical" decisions (use arc, assign job...).
- Polynomial number of vars and constraints.
- LP relaxation is often very weak.

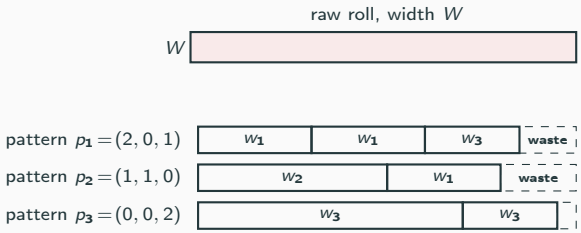
Extended (path-flow) formulation

- Variable per combinatorial object (route, pattern, roster).
- Possibly $\sim 10^{20}$ variables.
- LP relaxation is often much tighter.

Column generation lets us never write down the extended formulation.

Motivating example 1 – 1D Cutting Stock

Rolls of width W must be cut to satisfy demand d_i of width w_i.



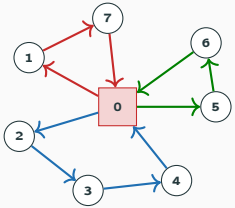
Pattern formulation (Gilmore–Gomory, 1961). $a_{ip} = \#$ items i in pattern p, with $\sum_i w_i a_{ip} \le W$. Variable $x_p \ge 0$ = "how many rolls cut according to p".

$$\min \sum_{p \in \mathcal{P}} x_p \quad \text{s.t.} \quad \sum_{p \in \mathcal{P}} a_{ip} x_p \ge d_i \ \forall i, \ x_p \in \mathbb{Z}_+.$$

The LP relaxation is empirically very tight (often within 1 unit of the IP optimum on 1D cutting-stock). We generate $\mathcal{P}$ on demand.

Motivating example 2 – VRPTW

Customers $\mathcal{N}$ with demand q_i, window $[a_i, b_i]$, depot 0, fleet of capacity Q.

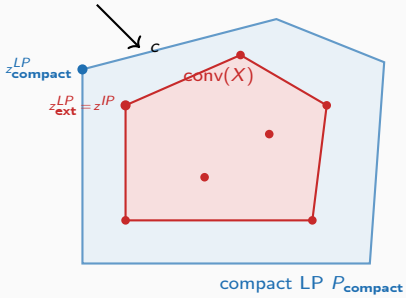


A *route* p = a feasible depot-to-depot tour visiting $S_p \subseteq \mathcal{N}$ inside capacity and windows. Cost c_p = total distance. 3 coloured routes shown = 3 columns of the master.

$$\min \sum_{p \in \mathcal{P}} c_p x_p \;\text{ s.t. }\; \sum_{p \in \mathcal{P}} a_{ip} x_p = 1 \;\forall i \in \mathcal{N},\; x_p \geq 0,\; a_{ip} = \mathbf{1}[i \in S_p].$$

Capacity & windows are pushed *inside* $\mathcal{P}$. $|\mathcal{P}| > 10^{20}$ on Solomon-100.

Why is the extended LP tighter?



The extended-formulation polytope is exactly $\text{conv}(X)$: the convex hull of the integer feasible objects (routes, patterns).

The compact LP is some outer relaxation of the same set – enlarged by all the convex combinations that the linking variables admit.

- Extended LP $\leq$ compact LP for any objective.
- Strict inequality whenever X has the non-integer extreme points that we have been hiding inside the pricer.

Concretely on VRPTW Solomon

Compact LP $\approx$ 60% of optimum; set-partitioning LP $\approx$ 99%. The difference between a 3-day B&C run and a 30-second B&P run.

Other classical applications

- **Crew scheduling / pairing** – columns are legal duties; pricing is a constrained shortest path on a flight network.
- **Nurse / employee rostering** – columns are individual rosters honouring labour rules.
- **Bin packing, multi-knapsack** – columns are packings.
- **Vehicle routing** (CVRP, VRPTW, PDPTW, DARP) – columns are routes.
- **Multi-commodity flow** – columns are paths per commodity.
- **Lot-sizing, network design, unit commitment** – columns are per-machine production plans / on-off schedules.
- **Graph colouring** – columns are independent sets.

Pattern

Whenever your problem looks like "assemble a global plan from many *individually feasible* sub-plans", think column generation.

When NOT to use column generation

- Compact LP relaxation is already close to optimum.
- No nice combinatorial structure in X (pricing problem is hopeless).
- Few enough columns that branch-and-cut handles them.
- You need fast *re-optimisation* after small data perturbations (CG warm-starts can be tricky – but possible).

Honest warning

CG is software-engineering heavy: master + pricer + branching + bounding + column pool management. A toy implementation in Python is doable in 300 lines (we will see); a competitive one is thousands.

Part 2 – The CG mechanics

The master problem (MP)

Generic LP with possibly exponentially many variables:

$$\text{(MP)} \quad z^* = \min_{x \geq 0} \sum_{p \in \mathcal{P}} c_p \, x_p \quad \text{s.t.} \quad \sum_{p \in \mathcal{P}} \mathbf{a}_p \, x_p \geq \mathbf{b}$$

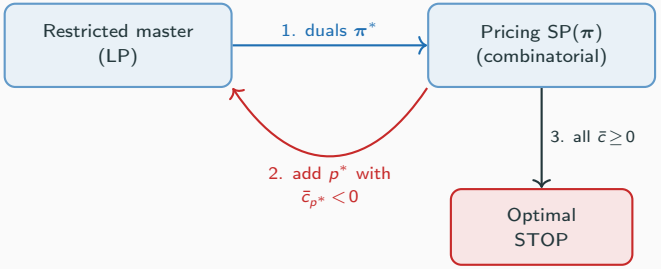
with $\mathbf{a}_p \in \mathbb{R}^m$, $c_p \in \mathbb{R}$, $|\mathcal{P}|$ huge.

Restricted master (RMP). Pick a small $\widetilde{\mathcal{P}} \subseteq \mathcal{P}$ containing at least one feasible point and solve

$$z^{\text{RMP}} = \min_{x \geq 0} \sum_{p \in \widetilde{\mathcal{P}}} c_p \, x_p \quad \text{s.t.} \quad \sum_{p \in \widetilde{\mathcal{P}}} \mathbf{a}_p \, x_p \geq \mathbf{b}.$$

Clearly $z^{\text{RMP}} \geq z^*$. We will iteratively add columns and tighten.

The basic loop, in one slide



1. Solve RMP $\rightarrow$ primal $\bar{x}$, duals $\boldsymbol{\pi}^*$.
2. Find p^* with $\bar{c}_{p^*} < 0$.
3. Else $\bar{x}$ optimal for MP.

Reduced cost of column p

$$\bar{c}_p \;=\; c_p \;-\; \boldsymbol{\pi}^{*\top}\mathbf{a}_p \;=\; c_p \;-\; \sum_i \pi_i^* a_{ip}.$$

A column with $\bar{c}_p < 0$ is undervalued by the current duals – bringing it into the basis improves the LP.

Reduced cost & optimality

Primal (RMP over $\widetilde{\mathcal{P}}$)

$$\begin{aligned} \min \ & \sum_{p \in \widetilde{\mathcal{P}}} c_p \, x_p \\ \text{s.t.} \ & \sum_{p \in \widetilde{\mathcal{P}}} \mathbf{a}_p \, x_p \ \geq \ \mathbf{b} \quad [\boldsymbol{\pi}] \\ & x_p \geq 0 \ \ \forall p \end{aligned}$$

Dual

$$\begin{aligned} \max \ & \boldsymbol{\pi}^\top \mathbf{b} \\ \text{s.t.} \ & \boldsymbol{\pi}^\top \mathbf{a}_p \ \leq \ c_p \ \ \forall p \in \widetilde{\mathcal{P}} \\ & \boldsymbol{\pi} \geq \mathbf{0} \end{aligned}$$

Each dual feasibility constraint is exactly $\bar{c}_p \geq 0$ with $\bar{c}_p = c_p - \boldsymbol{\pi}^\top \mathbf{a}_p$.

- If $\bar{c}_p \geq 0$ for all $p \in \mathcal{P}$ (not just in $\widetilde{\mathcal{P}}$), then $\boldsymbol{\pi}^*$ is dual feasible for MP and $\bar{x}$ is primal optimal – MP optimum reached.
- If some $\bar{c}_p < 0$, the dual constraint is violated by p: add p to $\widetilde{\mathcal{P}}$ and iterate.

Searching for violated dual constraints is the pricing problem SP: $\min_{p \in \mathcal{P}} \left\{ c_p - \boldsymbol{\pi}^{*\top} \mathbf{a}_p \right\}$.

Why "column" generation?

Imagine writing the (huge) constraint matrix A explicitly:

$$A = \begin{bmatrix} \mathbf{a}_{p_1} & \mathbf{a}_{p_2} & \cdots & \mathbf{a}_{p_K} \end{bmatrix}, \qquad K = |\mathcal{P}|.$$

The simplex method *prices* non-basic variables one by one until it finds one with negative reduced cost. CG just outsources that pricing to a combinatorial oracle:

	Standard simplex pricing	Column generation
Where	explicit columns	$\mathcal{P}$ implicit
Cost	$O(\lvert\mathcal{P}\rvert)$ scan	solve SP($\boldsymbol{\pi}$)
Output	one entering col.	one (or many) col.
Stop	all $\bar{c} \geq 0$	all $\bar{c} \geq 0$

What is the pricer for cutting stock?

Master min $\sum_p x_p$ s.t. $\sum_p a_{ip} x_p \geq d_i$, with dual $\pi_i \geq 0$ on each demand row.

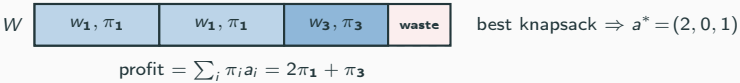
Question

A column p here is a cutting pattern $a_p \in \mathbb{Z}_+^I$ with $\sum_i w_i a_{ip} \leq W$. Its master cost is $c_p = 1$.

- Write the reduced cost $\bar{c}_p$ of pattern p.
- For fixed duals π, what classic combinatorial problem do you minimise to find the most negative $\bar{c}_p$?

Pricing for the pattern formulation (cutting stock)

Master $\min \sum_p x_p$ s.t. $\sum_p a_{ip} x_p \geq d_i$, dual $\pi_i \geq 0$ on each demand constraint. Reduced cost $\bar{c}_p = 1 - \sum_i \pi_i \, a_{ip}$. Pricing = integer knapsack $\max_{a \in \mathbb{Z}_+^I} \sum_i \pi_i a_i$ s.t. $\sum_i w_i a_i \leq W$.



- If profit > 1: pattern a^* has $\bar{c}_{p^*} < 0$, add it to RMP.
- If profit ≤ 1: no pattern improves RMP – LP optimum.

What is the pricer for VRPTW?

Master $\min \sum_p c_p x_p$ s.t. $\sum_p a_{ip} x_p = 1$, duals $\pi_i \in \mathbb{R}$. A column is a feasible route $p = (0, i_1, \ldots, i_k, 0)$, with $c_p = \sum_{(u,v)\in p} d_{uv}$.

Questions

- Write $\bar{c}_p$ as a sum over the arcs of the route.
- What is the resulting pricing problem on the customer graph? How is it different from a classical shortest path?

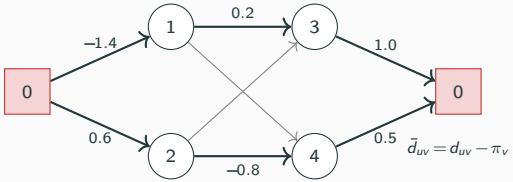
Hint: think about what the duals π_v "pay" along the route.

Pricing for the set-partitioning master (VRPTW)

Master $\min \sum_p c_p x_p$ s.t. $\sum_p a_{ip} x_p = 1$, duals $\pi_i \in \mathbb{R}$. A route $p = (0, i_1, \ldots, i_k, 0)$ has $c_p = \sum_{(u,v)\in p} d_{uv}$ and visits S_p, so

$$\bar{c}_p = c_p - \sum_{i \in S_p} \pi_i = \sum_{(u,v)\in p} \big(d_{uv} - \pi_v\big), \ \pi_0 := 0.$$

Pricing = shortest depot-to-depot path on the graph with arc cost $\bar{d}_{uv} = d_{uv} - \pi_v$, subject to capacity & time windows: RCSPP.



- Negative arc costs are normal – they reflect "well-paid" customers under the current dual prices.
- Capacity, time, elementarity are resources along the path.

First example end-to-end – toy cutting stock

$W = 10$, items $w = (3, 5, 7)$, demands $d = (20, 10, 5)$.


```
master.addColumn(p1=(3,0,0), cost=1)                # covers item 1
master.addColumn(p2=(0,2,0), cost=1)                # covers item 2
master.addColumn(p3=(0,0,1), cost=1)                # covers item 3
pi = master.solve_dual()                            # ~ (1/3, 1/2, 1)
p_new = knapsack(profits=pi, weights=(3,5,7), W=10)    # -> (1,0,1)
master.addColumn(p_new, cost=1)                     # red. cost 1 - (1/3 + 1) < 0
```

After a few iterations the LP settles at $z^{LP} = 13.5$; $\lceil 13.5 \rceil = 14$ is the IP optimum.

Why no upper bound on x_p?

The LP we solve is

$$\min \sum_p c_p x_p \quad \text{s.t.} \quad \sum_p a_{ip} x_p \; [=\text{or}\geq] \; b_i \;\; \forall i, \quad x_p \geq 0.$$

We only declare $x_p \geq 0$ – never an upper bound $x_p \leq 1$.

- The covering / partitioning constraint already bounds x_p: in cutting stock $\sum_p x_p \leq \sum_p a_{ip} x_p / a_{ip}$ and in VRPTW $x_p \leq \sum_p a_{ip} x_p = 1$ when the customer $i \in S_p$ exists. Bounds "built in".
- Adding $x_p \leq 1$ creates an extra dual $\mu_p \geq 0$ on each column. The pricer would then need to compute $\bar{c}_p = c_p - \boldsymbol{\pi}^\top \mathbf{a}_p + \mu_p$ – but μ_p depends on the column we're trying to find. That's a chicken-and-egg pricing problem.
- Dropping the upper bound also keeps the master sparse and lets simplex avoid degeneracy from artificial bound constraints.

Rule

Continuous LP master: $x_p \geq 0$, no upper bound. Integrality ($x_p \in \{0, 1\}$) is only re-imposed inside heuristics or B&P.

Initial columns – bootstrap

RMP must be feasible from iteration 1. Two go-to recipes:

- **One column per covering constraint.** For VRPTW, one singleton route depot $\to i \to$ depot for each customer $i \in \mathcal{N}$ – always feasible, gives a valid finite dual. For cutting stock, one "one-item" pattern per item type. This is what the lab does.
- **Slack / artificial variables.** Add a per-constraint slack $s_i \geq 0$ with huge cost M in the master objective. Always feasible regardless of the column pool. Drop slacks (or watch them drift to 0) as real columns enter.

Rule of thumb

Either start with one route/pattern per customer/item, or add a slack column per constraint – but never start empty.

Why not start empty?

Initial columns – bootstrap

RMP must be feasible from iteration 1. Two go-to recipes:

- **One column per covering constraint.** For VRPTW, one singleton route depot $\to i \to$ depot for each customer $i \in \mathcal{N}$ – always feasible, gives a valid finite dual. For cutting stock, one "one-item" pattern per item type. This is what the lab does.
- **Slack / artificial variables.** Add a per-constraint slack $s_i \geq 0$ with huge cost M in the master objective. Always feasible regardless of the column pool. Drop slacks (or watch them drift to 0) as real columns enter.

Rule of thumb

Either start with one route/pattern per customer/item, or add a slack column per constraint – but never start empty.

Why not start empty?

With an infeasible RMP the dual is unbounded: the pricer gets bogus prices and convergence is unreliable.

Quick view – Dantzig–Wolfe decomposition

Take any LP with block-diagonal structure

$$\min c^\top x \text{ s.t. } A_0 x \geq b_0,\ x \in X = \{x : Bx \geq d,\ x \geq 0\}.$$

X a polyhedron $\Rightarrow$ every $x \in X$ writes as $x = \sum_p \lambda_p v_p + \sum_r \mu_r r_r$, $\lambda \geq 0, \sum_p \lambda_p = 1$, $\mu \geq 0$ (vertices v_p, rays r_r). Substituting:

$$\min \sum_p (c^\top v_p)\lambda_p + \sum_r (c^\top r_r)\mu_r \text{ s.t. } \sum_p (A_0 v_p)\lambda_p + \sum_r (A_0 r_r)\mu_r \geq b_0,\ \mathbf{1}^\top \lambda = 1\ \lambda, \mu \geq 0.$$

This is the master problem; one variable per vertex/ray of X. Pricing = optimisation over X with modified objective $c - \pi A_0$.

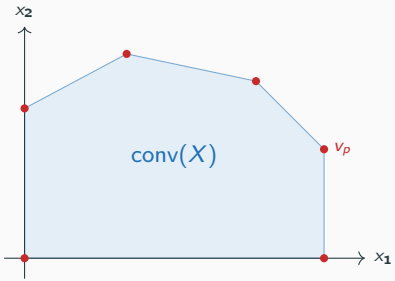
Why we are not deriving DW today

- In our applications X is implicit (set of routes, set of patterns) and we go straight to the path/pattern formulation – no block structure to convexify on paper.
- DW gives the same master/pricing pair, with a clean formal status for the pricing oracle and the Lagrangian dual.
- Useful when you read the literature: Lagrangian relaxation of A_0, column generation and DW are all the same algorithm with different accents.

One-line summary

Whether you start from "the LP has too many columns" (CG view) or from "the LP has block structure that we want to push into a subproblem" (DW view), you get the same RMP–SP loop.

Geometric picture



Each red dot is a feasible *route* / *pattern*. The master optimises a linear function on the convex hull of these dots, intersected with the linking constraints $A_0 x \geq b_0$.

Pricing returns a vertex v_p that improves the objective; it never enumerates the dots, it walks on X directly.

Termination, in theory and in practice

- There are finitely many extreme points in X, so CG terminates after finitely many iterations.
- In practice "finitely" can be thousands of iterations on large instances.
- Last 5% of iterations often produce columns that contribute almost nothing – the famous tailing-off effect (Part 4).

Stopping criteria you'll see

- Exact: pricer returns no negative-reduced-cost column.
- Heuristic: RMP improvement below threshold ε for K consecutive iterations.
- Bound-based: Lagrangian lower bound $\geq$ best known IP minus δ%.

Exercise A – Build the master in HiGHS

Exercise A – master & initial columns

- Open `lab/python/vrp/cg/master_problem.py`.
- Implement `add_constraints` (one $=1$ row per customer) and `set_objective` (sum of route costs).
- In `vrp.py`, implement `generate_initial_paths`: one round-trip route per customer.
- Run `python -m vrp.tests.toy_master` and check z_0^{RMP} matches the expected value printed in the test.

Time budget: 15 minutes.

Part 3 – The pricing subproblem (RCSPP)

What is RCSPP?

Given a digraph $G = (V, A)$ with arc costs c_{uv} and resource consumptions $\rho_{uv}^{(r)}$ for $r = 1, \dots, R$, plus per-resource bounds $[\ell_v^{(r)}, u_v^{(r)}]$ at each node:

$$\min \sum_{(u,v)\in P} c_{uv} \quad \text{s.t.} \quad P \text{ is an } o \to d \text{ path with } \ell^{(r)} \le \mathcal{R}^{(r)}(P) \le u^{(r)} \ \forall r.$$

$\mathcal{R}^{(r)}(P)$ = accumulated resource r along P (e.g. time, load).

- $R = 0 \Rightarrow$ classical shortest path (Dijkstra/Bellman).
- Non-trivial resources $\Rightarrow$ NP-hard in general.
- In CG we have negative arc costs (because of duals), so we can't use Dijkstra naively even when there are no resources.

Resources, the workhorse abstraction

A resource is any quantity that

- starts at some value at the origin,
- is updated along the path by an extension function $f_{uv}^{(r)} : \mathbb{R} \to \mathbb{R}$,
- must satisfy a feasibility window $[\ell_v^{(r)}, u_v^{(r)}]$ at every node v,
- supports a dominance relation for label pruning.

Examples in VRPTW:

Resource	Update	Window at v	Dominance
Distance / cost	$T' = T + c_{uv}$	$(-\infty, +\infty)$	$\leq$
Time	$T' = \max(T + s_u + t_{uv}, a_v)$	$[a_v, b_v]$	$\leq$
Load	$Q' = Q + q_v$	$[0, Q^{\max}]$	$\leq$
Visited set	$S' = S \cup \{v\}$	elementary, $v \notin S$	$\subseteq$

In our `rcspp` package these are encoded by the `ExtensionFunction`, `FeasibilityFunction`, `DominanceFunction` triple.

Label-setting algorithm (Desrochers & Soumis, 1988)

A label at node v is a tuple $L = (v,\ \text{cost},\ \rho^{(1)}, \ldots, \rho^{(R)},\ \text{predecessor})$.

Generic label-setting

1: Initialise: at origin o, push label L_0 with cost 0 and resources at lower bounds.
2: while unprocessed labels exist
3: pick label L at node u (priority: cost or lex. on resources)
4: for each arc $(u, v) \in A$
5: extend $L \to L'$ via f_{uv}
6: if L' feasible and not dominated, store it
7: dominate / discard newly dominated labels at v
8: Best label at destination d = optimal RCSPP path

Correctness: dominance + monotone extensions $\Rightarrow$ no useful label is ever discarded.

Dominance, the trick that keeps it tractable

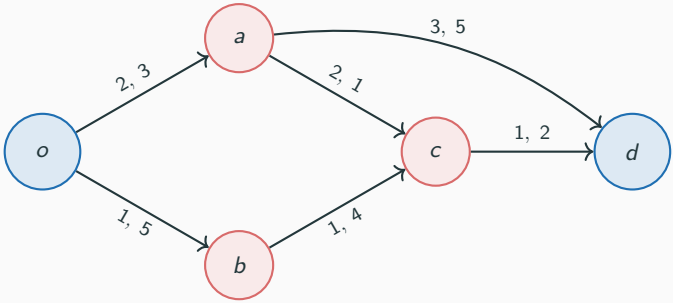
Label L_1 dominates L_2 at the same node v if

$$\text{cost}(L_1) \leq \text{cost}(L_2), \quad \rho^{(r)}(L_1) \leq \rho^{(r)}(L_2) \ \ \forall r,$$

with at least one strict inequality. Then any feasible completion of L_2 is also feasible from L_1 and cheaper.

- Pruning power scales with the number of partially ordered resources.
- Elementarity (set of visited customers) is *not* totally ordered $\Rightarrow$ dominance is much weaker once we enforce it.
- Practical compromise: ng-routes (Baldacci et al., 2011) – forbid revisits only inside small neighbourhoods N_v.

A worked RCSPP example



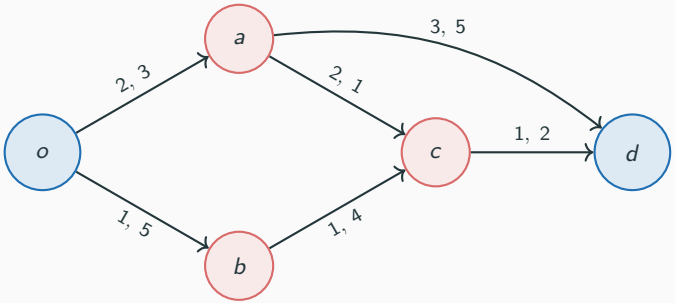
Path	$\sum c$	$\sum \rho$
$o \to a \to c \to d$		
$o \to b \to c \to d$		
$o \to a \to d$		

Optimum:

Each arc labelled $(c_{uv},\ \rho_{uv})$. Resource budget at d: $\rho(P) \leq 7$.

Shortest path and feasible path are not the same.

A worked RCSPP example



Path	$\sum c$	$\sum \rho$
$o \to a \to c \to d$	5	6 ✓
$o \to b \to c \to d$	3	11 ×
$o \to a \to d$	5	8 ×

Optimum: $o \to a \to c \to d$, cost 5.

Each arc labelled $(c_{uv},\ \rho_{uv})$. Resource budget at d: $\rho(P) \leq 7$.

Shortest path and feasible path are not the same. The cheaper $o \to b \to c \to d$ violates the resource budget.

Elementary or not? coefficient a_{ip} in the master

Two consistent ways to describe a route p in the set-partitioning master:

- **Elementary**: every customer appears at most once, $a_{ip} = \mathbf{1}[i \in S_p] \in \{0, 1\}$. Coefficient is binary.
- **Non-elementary** (RCSPP): pricer may produce routes revisiting a customer; we set $a_{ip} = \#$ visits of i in p, which can be ≥ 2.

Both formulations are mathematically valid. The set-partitioning master enforces $\sum_p a_{ip} x_p = 1$ in either case; with $a_{ip} = 2$ for a route that visits i twice, the LP simply needs to set $x_p = 1/2$ or pay heavily.

- Elementary: tighter LP, harder pricer (ERCSPP).
- Non-elementary: weaker LP (revisits allowed at LP level), much easier pricer (RCSPP).
- ng-routes interpolate: cheap pricer, almost-elementary LP.

ERCSPP vs RCSPP

ERCSPP	visit each node at most once; pricing for set-partitioning master with $= 1$ constraints.
RCSPP	nodes may be revisited; pricing for set-covering (≥ 1).

- ERCSPP is strongly NP-hard.
- RCSPP with non-negative reduced costs and integer resources is solvable in pseudo-polynomial time.
- Most modern VRPTW solvers price ng-routes: between RCSPP and ERCSPP, they tighten dominance compared to RCSPP but stay tractable.

Practical advice

Avoid solving the full ERCSPP in general – it explodes combinatorially. Start with RCSPP pricing; if the master ends up with too many revisiting routes, switch to ng-routes. Reserve exact ERCSPP for small instances or for the last few branch-and-price nodes.

Heuristic vs exact pricing

Method	Cost per call	Returns
Greedy / nearest-neighbour	$O(n^2)$	1 negative-cost route
Heuristic label-setting	$O(n \cdot \lvert A \rvert)$	a handful
Exact label-setting (RCSPP / ERCSPP)	exp. in worst case	all neg. red. cost
Bidirectional w. join	faster than fwd	same

- During the first iterations, nobody calls the exact pricer.
- Hierarchy: greedy $\rightarrow$ heuristic *to* exact RCSPP / ERCSPP.
- For LP optimality, the last pricing call must be exact.
- For the Lagrangian bound (Part 4), we also need a true lower bound from the pricer – heuristic pricers are not enough.

Returning multiple columns per call

CG converges much faster if each pricing call adds many columns.

- Bidirectional / multi-label algorithms naturally produce many non-dominated sink labels with $\bar{c} < 0$.
- Filter: keep only the K best, or the most "diverse" (different customer subsets).
- Trade-off: more columns $\Rightarrow$ heavier RMP.
- Typical $K \in \{20, 50, 100\}$ on Solomon-100.

In the lab, `VRP(K_MAX=K)` caps the number of columns added per CG iteration. The `compare_cg` runner sweeps K for you.

The rcspp Python package, in 30 seconds

```
from rcspp.graph     import ResourceGraph
from rcspp._core.graph import Row
from rcspp.resource import (
    AdditionExtensionFunction,              # cost or load
    ValueDominanceFunction,                 # <= dominance
    ValueCostFunction,
    TrivialFeasibilityFunction,             # no bound
    MinMaxFeasibilityFunction,              # [lo, hi]
    TimeWindowExtensionFunction,            # max(T+travel, a_v)
    TimeWindowFeasibilityFunction,          # T <= b_v
)
g = ResourceGraph()
g.add_real_resource(ext, feas, cost, dom) # 1 call / resource
g.add_node(node_id, is_source, is_sink)
g.add_arc((dist, time, load),             # base resource incr.
          o, d, arc_id, dist,             # base cost = dist
          [Row(o, 1.0)])                  # 1*pi_o subtracted
g.update_reduced_costs(dual_by_id)        # cheap re-pricing
solutions = g.solve()                     # Solution[] sorted with negative reduced cost
```

- Each arc carries base costs (no duals baked in) plus a list of dual `Rows`. The reduced cost is $\bar{c}_{uv} = c_{uv}^{\text{base}} - \sum_r \alpha_r \pi_{i_r}$.
- Build the graph once; call `update_reduced_costs` every CG iteration.

Exercise B – Wire the pricing subproblem

Exercise B – arc costs & RCSPP solve

- In `vrp.py`, complete `add_arc_to_graph`: arc cost $= d_{uv} - \pi_v$ (with $\pi_0 := 0$), three resource increments (cost, time, demand), in the same order as the `add_real_resource` calls in `construct_resource_graph`.
- Replace the body of `solve_subproblem` so that it constructs the resource graph with the latest duals and returns the result of `ResourceGraph.solve()`.
- Run `python -m vrp.tests.toy_pricing` and check that the first negative-reduced-cost route on `toy.txt` matches the expected route printed by the test.

Time budget: 25 minutes.

Part 4 – Making column generation fast

Where the time goes

Profile of a typical CG iteration on VRPTW:

- Rule of thumb: pricing is 70-95% of total time, should never be less than 40%.
- LP solve grows roughly linearly with the number of accumulated columns (warm-started simplex / interior point reuse).
- Memory: the column pool can balloon to millions of routes – you will need a pruning strategy.

Trick 1 – Multi-column pricing (revisited)

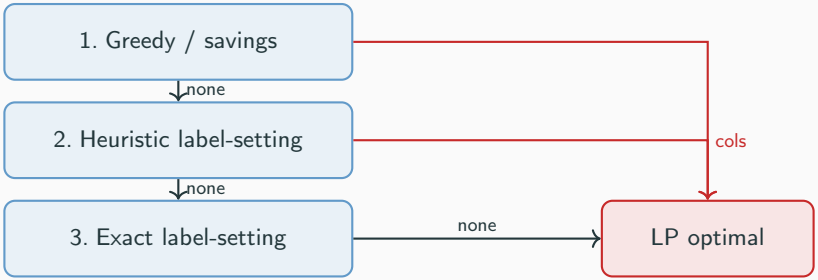
Add many columns per iteration to amortise the pricing.

- "All non-dominated sink labels with $\bar{c} < -\varepsilon$"
- or top-K by $\bar{c}$,
- or one column per "corridor" of the graph (diversity).

Effect: 5-30% iteration count reduction, often 2x wall-clock. Not a strict free lunch – on degenerate masters it can amplify cycling.

Trick 2 – Hierarchy of pricers

Inside the same path family, escalate cheap → expensive only when the cheap one finds no negative-reduced-cost column:



- All three solve the *same* pricing problem – they differ only in how thoroughly they search.
- For LP optimality the last call must be exact.
- The exact pricer is typically only used at the end of the root and at every B&P node closure.

Path family is a separate choice

Don't confuse the algorithmic hierarchy above with the choice of path family, which decides what counts as a feasible column:

Family	Definition / pricing problem
RCSPP	nodes can be revisited; coefficient a_{ip} in master can be ≥ 2. Pseudo-poly with non-negative duals.
ng-routes	nodes can be revisited only if the recently visited set does not contain them. "Almost elementary", partial dominance.
ERCSPP	every customer visited at most once. Strongest LP, pricer is strongly NP-hard.

- These are not levels you escalate through during one CG run – you pick one (typically ng-routes) and stay with it.
- The only time you switch is to fix specific routes that visit customers more than once (Ryan-Foster, Part 6).

Trick 3 – Stabilisation

Without stabilisation, dual prices oscillate wildly $\Rightarrow$ pricer produces useless or duplicate columns.

Three popular schemes:

- Box-step / trust region: bound the change in $\boldsymbol{\pi}$.
- Bundle / proximal: quadratic penalty on $\|\boldsymbol{\pi} - \hat{\boldsymbol{\pi}}\|^2$.
- Dual-price smoothing (Wentges): $\boldsymbol{\pi}^{(k+1)} = \alpha\, \boldsymbol{\pi}^{(k)} + (1-\alpha)\, \boldsymbol{\pi}^{(k)}_{\text{RMP}}$ with $\alpha \in [0, 1)$.
- Du Merle penalties: artificial dual bounds via penalty variables in the master.

Practical advice

Wentges smoothing is one extra line of code, makes a huge difference, and converges to the same optimum for $\alpha < 1$. Start there.

Trick 4 – Handling the column pool

- Keep all columns generated; they will be needed at branching.
- Track each column's age (= iters since last positive in RMP).
- In the master LP, only *warm-start* with active columns; pool stays on disk / in a side structure.
- At a branching decision, only columns *compatible* with the branch are passed on (filter by Ryan-Foster or arc-branch criterion).

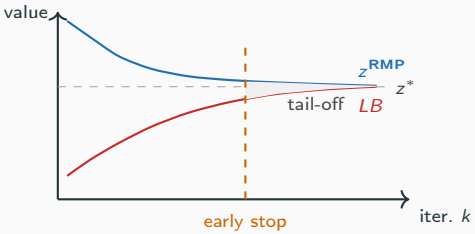
Trick 5 – Fast LP re-optimisation

- Adding columns is a trivial re-optimisation: simplex stays dual feasible, primal-feasible after a few pivots.
- Use the primal simplex after a column add (fewer pivots than dual).
- Conversely, after adding a constraint or branching, dual simplex is the right re-optimisation.
- HiGHS, Clp, Gurobi all expose explicit "add column" APIs – avoid rebuilding the model.

Beware

Building the LP from scratch each iteration is terrible for production. In the exercice, you will keep the LP alive across iterations.

Trick 6 – Tailing-off and early termination



- z^{RMP} decreases very slowly at the end (tailing-off).
- The Lagrangian bound LB (Trick 7) only catches up on the last few iterations.

Early stop when

- rel. LP improvement $< \varepsilon$ for K iters,
- LB stagnates within tolerance, or
- time / iteration budget exhausted.

Stopping early kills z^{RMP} as a bound; LB stays valid.

How do we bound the LP early?

At iteration k, the master value z^{RMP_k} is an upper bound on the LP – it can only decrease as we add columns.

Question

- How do we get a lower bound on z^* *before* CG converges?
- What information do we already have at iteration k? (Hint: we just solved the pricing optimum exactly.)

Answer on the next slide – it's the Lagrangian bound.

Trick 7 – The Lagrangian lower bound

At any CG iteration with current dual $\boldsymbol{\pi}$ and exact pricing optimum $\bar{c}^* = \min_{p \in \mathcal{P}} \; c_p - \boldsymbol{\pi}^\top \mathbf{a}_p$,

$$z^* \;\geq\; \boldsymbol{\pi}^\top b \;+\; K \cdot \min(0,\, \bar{c}^*) \;=:\; \mathrm{LB}(\boldsymbol{\pi}),$$

where K is the number of vehicles and b is the master RHS vector.

- For VRPTW with $b_i = 1$: $LB = \sum_{i \in \mathcal{N}} \pi_i + K\, \bar{c}^*$.
- Valid at every iteration, unlike z^{RMP} which is a true bound only at LP convergence.
- This requires the master to include the vehicle-count constraint $\sum_p x_p \leq K$ – without it, the Lagrangian relaxation is unbounded below.
- Heuristic pricers do not give a valid LB; only the exact pricer does.

The bound in 4 lines of Python

```python
def lagrangian_bound(mp_dual_by_id, sigma, pricing_sols, K):
    """LB = sum(pi_i) + K*sigma + K*min(0, min_redcost)."""
    pi_sum = sum(mp_dual_by_id.values())
    redc   = min((s.cost for s in pricing_sols), default=0.0)
    return pi_sum + K * sigma + K * min(0.0, redc)
```

- `sigma` is the dual on the vehicle-count constraint.
- Print this every iteration: it must increase (with smoothing) and end equal to z^{RMP} at LP optimum.
- Use it as the stop rule at the root and as the prune rule in B&P.

Two new ways to stop CG early (lab `gap_pct` & `incumbent_ub`)

In the lab, `VRP.solve_cg(incumbent_ub, gap_pct)` can stop before pricing returns $\bar{c}^* \geq 0$:

- **UB cut** – when $LB \geq UB - \varepsilon$, the subtree is already proved optimal: no point in adding more columns. B&P passes the current incumbent down to each child's CG run.
- **Gap cut** – when $(z^{\text{RMP}} - LB)/|z^{\text{RMP}}| \leq \delta\%$. Useful at the root for an approximate δ-optimal LP.

Why a gap stop is acceptable

A solver run with `gap_pct=2` returns z^{RMP} and an LB that bracket z^* within 2% – still a certified bound, even if incomplete. Often saves the last 80% of iterations spent in tailing-off.

Trick 8 – Symmetry and elementarity

- In identical-vehicle VRP, set-partitioning is naturally symmetry-free (one variable per route, regardless of vehicle).
- Cutting stock: identical rolls disappear in pattern formulation.
- Watch out for accidentally symmetric objectives: if your route cost c_p does not depend on which vehicle uses it, drop vehicle indices everywhere.
- Elementarity vs RCSPP: revisits are LP-feasible only when the dual variables of revisited customers are positive enough to make the loop "profitable". ng-routes give you 90% of elementarity for 10% of the work.

Putting it all together (efficient CG loop)

Efficient column generation

paths ← initial columns (heuristic)
LP ← build RMP on paths
iter ← 0
repeat
  solve LP ⇒ duals π
  $\pi \leftarrow$ smooth(π^{prev}, π) # Wentges
  $S \leftarrow$ greedy_pricer(π);
    if $S = \varnothing$: $S \leftarrow$ ng_pricer
    if $S = \varnothing$: $S \leftarrow$ exact
  if $S = \varnothing$: break
  keep top-K columns from S, add to LP (warm start)
  compute LB; if $z^{\text{RMP}} - \text{LB} \leq \delta$: break
end repeat

Exercise C – multi-column, smoothing, LB

Exercise C – efficient CG

- Cap the number of columns added per iteration with the parameter `K_MAX`; try $K \in \{1, 5, 20, 100\}$ on `R101_25.txt` and record total iterations & wall-clock.
- Wentges smoothing: $\boldsymbol{\pi} \leftarrow \alpha \boldsymbol{\pi}^{prev} + (1 - \alpha)\boldsymbol{\pi}$, $\alpha \in [0.3, 0.7]$.
- Implement `lagrangian_bound(...)` (Trick 7) and print it every iteration. Stop CG when $z^{\text{RMP}} - LB \leq \delta$.
- Don't rebuild the master from scratch: each CG iteration just calls `master.add_column(...)` on the new columns, then re-solves the LP (warm-started simplex).

Time budget: 25 minutes.

Why we don't rebuild

Adding a column is a primal-feasible perturbation: simplex needs only a handful of pivots. Rebuilding from scratch throws the basis away and is the single biggest performance hit we'll see.

Part 5 – From LP to integer (heuristics)

The LP is solved – now what?

- Solving the LP relaxation of the master gives a fractional $\bar{x} \in [0,1]^{|\bar{\mathcal{P}}|}$.
- Even if $\bar{x}$ is integer feasible, that is rare in real instances.
- Two families of next steps:
  - Heuristic: get a good integer solution fast, no optimality guarantee.
  - Exact: branch-and-price (Part 6): Branch-and-bound on the master IP, where every node's LP relaxation is solved by column generation.
- Today we will use both: heuristics give a working incumbent in the lab, and we will then plug it into a small branch-and-price loop (Part 6 + Exercise E) that closes the optimality gap with the Lagrangian bound.

Heuristic 1 – Restricted Master Heuristic (RMH)

Once CG terminates at the root, freeze the column pool $\widetilde{\mathcal{P}}$ and solve

$$z^{\text{RMH}} \;=\; \min \sum_{p \in \widetilde{\mathcal{P}}} c_p \, x_p \;\; \text{s.t.} \;\; A\,x \geq b, \;\; x_p \in \{0,1\}.$$

- Just hand the pool to a MIP solver (HiGHS, CBC, Gurobi).
- Quality depends entirely on the column diversity in the pool.
- Fast on small/medium VRP, often within 1-2% of optimal on Solomon-25.
- Will be the first integer solver we plug in the lab.

Caveat

The pool is generated for *LP* optimality, not for integrality. On hard instances RMH can be feasibility-infeasible – solve a set-covering relaxation as a fallback.

Heuristic 2 – Diving (price-and-fix)

Branch-and-bound with no backtrack and no bound checks.

Diving heuristic

1: solve root CG; let $\bar{x}$ be its LP solution
2: repeat
3: pick column p^* with $\bar{x}_{p^*}$ closest to 1 (or largest)
4: fix $x_{p^*} = 1$
5: re-run CG on the residual master (warm-started)
6: re-solve LP $\Rightarrow$ new $\bar{x}$
7: until master integer (or infeasible)

- Each fix shrinks the residual problem (covered customers are locked).
- Convergence is fast: at most $|\mathcal{N}|$ fixes.
- Often beats RMH because it drives new compatible columns at each fix.

Diving variants

- **Multi-fix diving.** Fix several columns at once each iteration – faster, lower quality.
- **LP-rounding dive.** Round $\bar{x}$ aggressively then resolve LP; cheap warm-start.
- **Limited-discrepancy diving (LDS).** Allow at most K "mistakes" along the dive – a mistake is a fix that disagrees with the LP recommendation (e.g. second-best column instead of the closest-to-1). Equivalent to a depth-first search bounded by discrepancy budget K. Catches situations where the LP advice was wrong on one or two early fixes; keeps the runtime essentially linear in K.
- **Restart diving.** Re-run dive from new random seeds; pick best.

For the lab

Implement plain diving first: pick column with largest $\bar{x}_p$, fix to 1, re-run CG. 30 lines.

Heuristic 3 – Price-and-branch (no bounding)

- Run B&P but do not prune by bound: keep only the feasibility cuts (drop branches that the master proves infeasible). The Lagrangian / LP bound is computed but not used for pruning.
- Effect: the search becomes a heuristic "deep tree" that follows fractional structure but never gives up on a node because of bound.
- Combined with depth-first search and a good incumbent heuristic (RMH at every node), this finds high-quality integer solutions quickly without exploring the whole tree.
- Often used as the warm-start of the exact B&P – the incumbent it produces tightens pruning later.

Hierarchy of heuristics in a serious solver:

$$\text{RMH} \longrightarrow \text{LP-rounding} \longrightarrow \text{plain diving} \longrightarrow \text{LDS dive} \longrightarrow \text{full B\&P.}$$

When the heuristic is enough

- Operational use cases (daily routing): a 1-2% gap heuristic, run inside a refresh budget of a few minutes, is usually preferable to an overnight optimum.
- Tactical / strategic studies (yearly fleet design): you want the true optimum – branch-and-price.
- **Sensitivity analyses.** If you need the marginal value of servicing customer i (e.g. to decide whether to insert a new order, to negotiate price, or to compute opportunity costs across scenarios), you read it off the master dual π_i. Heuristic dives produce duals coming from intermediate restricted masters – they are not the true LP duals of the original problem and can shift by 30–50% between two consecutive dives. For reliable parametric output:
  - solve the root CG to LP optimality, take its duals (*RMH at root only*), or
  - run full B&P and read duals at the optimal node.

  Anything in between is fine for an objective value but unsafe for marginal-cost reasoning.

Exercise D – Heuristic IP on the column pool

Exercise D – RMH then plain dive

- After CG converges at the root, take the column pool and call `MasterProblem.solve(relax=False)`: that's the Restricted-Master Heuristic. Record incumbent value.
- Implement plain diving: pick the column p^* with largest $\bar{x}_{p^*}$, set its upper bound to 1 in the master, re-run CG, repeat until integer or infeasible.
- Compare RMH vs. diving on `R101_25.txt` – which gives the better incumbent? Which is faster?

Time budget: 25 minutes.

Part 6 – Branch-and-Price

Branch-and-price in one sentence

Branch-and-bound on the master IP, where every node's LP relaxation is solved by column generation.

Three things change between B&B and B&P:

1. The LP at each node is huge $\Rightarrow$ solve it by CG.
2. Branching on a master variable x_p destroys the pricing structure $\Rightarrow$ branch on something else.
3. Every node needs valid columns; the column pool must be filtered to remain compatible with the branch.

How would you branch?

Imagine the LP returns $\bar{x}_{p_1} = 0.5$, $\bar{x}_{p_2} = 0.5$, $\bar{x}_{p_3} = 0$, ... with p_1 and p_2 visiting partly different customers.

Questions

- Why does $x_p \leq 0$ vs $x_p \geq 1$ (branching on the master variable) destroy the branch-and-bound tree and the pricer?
- Find a binary criterion on *customer pairs* that always shows fractionality whenever $\bar{x}$ is fractional.

The naive branch (don't do this)

Just branching on a fractional master variable

$$x_p = 0.5 \implies \text{down: } x_p \leq 0, \quad \text{up: } x_p \geq 1.$$

- Up-branch: fine – $x_p \geq 1$ locks column p in the incumbent; the residual master is solved by CG with the locked column counted in the constraint slack.
- Down-branch: forbid one specific column p. The pricer needs to enumerate "all paths except p" – equivalent to a combinatorial k-shortest-path problem, which is much harder than plain RCSPP and breaks dominance.
- The tree is also extremely unbalanced (one excluded column among 10^{20}).

Take-away

Never branch on the master variables themselves.

Ryan-Foster branching (set-partitioning case)

Set-partitioning constraints $\sum_p a_{ip} x_p = 1$. If $\bar{x}$ is fractional, there must exist customers i, j such that

$$\sum_{p:\,(i,j)\in S_p} \bar{x}_p \;\in\; (0,1),$$

i.e. i and j are sometimes together, sometimes not.

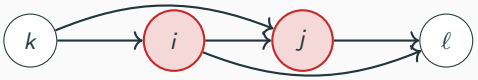
Two-way branch:

- **Together**: keep only routes that visit both or neither. Implement by *merging* i and j into a single node with combined demand and the time window $[a_i, b_i] \cup [a_j, b_j]$ (the union of the windows).
- **Apart**: forbid routes that visit both. Implement by deleting all $i \to \ldots \to j$ paths from the pricing graph.

Pricing structure is preserved on both branches.

Ryan-Foster, the picture

Top: original pricing graph. Bottom-left: *together*-branch. Bottom-right: *apart*-branch.



original graph

TOGETHER

"heuristic" merge i, j into one node;
$q = q_i + q_j$, window "$[a_i, b_i] \cup [a_j, b_j]$"

APART

forbid every $i \rightsquigarrow j$ subpath
(red dashed arcs deleted)

Key property

Both branches modify only the pricing graph; the pricer code, dominance and resources are unchanged.

Branching on arcs / flow

Define the aggregated arc flow

$$y_{uv} = \sum_{p\,:\,(u,v)\in p} x_p.$$

If $\bar{y}_{uv} \in (0,1)$, branch on $y_{uv} \leq 0$ vs $y_{uv} \geq 1$.

- Down-branch: remove arc (u, v) from pricing graph.
- Up-branch: force arc (u, v) – contract endpoints into a single super-node with merged time window.
- Universally compatible with VRP, scheduling, network design.
- Often produces more balanced trees than Ryan-Foster, but deeper.

Branching on resources / time / load

When neither Ryan-Foster nor arc branching gives fractionality, branch on a resource:

$$\tau_v \;=\; \sum_p t_v(p)\,\bar{x}_p \quad \text{(expected start time at } v\text{)}.$$

Branch $\tau_v \le \lfloor \bar{\tau}_v \rfloor$ vs $\tau_v \ge \lceil \bar{\tau}_v \rceil$.

- Implemented by splitting the time window of v into two halves between branches.
- Same trick for vehicle load, working hours, etc.
- Useful late in the tree when only a few customers remain fractional.

Modern recipe

Combine: arc branching first, Ryan-Foster on tied customers, resource branching for the remainder. All three preserve pricing structure.

The Lagrangian bound (Lasdon, 1970; Vanderbeck, 2006)

Master LP value z^{RMP} is not a valid lower bound until CG converges. The Lagrangian bound is a valid lower bound at every CG iteration.

Let $\boldsymbol{\pi}$ be the current dual and let $\bar{c}^* = \min_{p \in \mathcal{P}} c_p - \boldsymbol{\pi}^\top \mathbf{a}_p$ be the pricing optimum. Then for the master MP with convexity (one vehicle/object):

$$z^* \;\geq\; \boldsymbol{\pi}^\top b \;+\; \bar{c}^* \quad \text{(Lagrangian dual at } \boldsymbol{\pi}\text{)}.$$

For multi-vehicle / multi-roll, with K identical resources:

$$z^* \;\geq\; \boldsymbol{\pi}^\top b \;+\; K \cdot \bar{c}^*.$$

Combine with the master LP: $z^* \leq z^{\text{RMP}}$, but at any time $z^* \geq \boldsymbol{\pi}^\top b + K\bar{c}^*$.

Why the Lagrangian bound matters

- It monotonically increases (with the right post-processing) and gives a valid stopping rule before LP convergence.
- Plug it into B&P node pruning – close branches whose Lagrangian bound exceeds the incumbent before paying for full LP convergence.
- Required ingredient for the integrated dual stabilisation + bound machinery in modern solvers (BaPCod, VRPSolver, GCG).
- In our lab, you'll print it every iteration and watch it close in on the LP.

Subtle point

The bound uses $K \cdot \bar{c}^*$ *only* if the pricer's $\bar{c}^*$ is a true minimum – heuristic pricers don't give a valid bound.

The B&P algorithm template

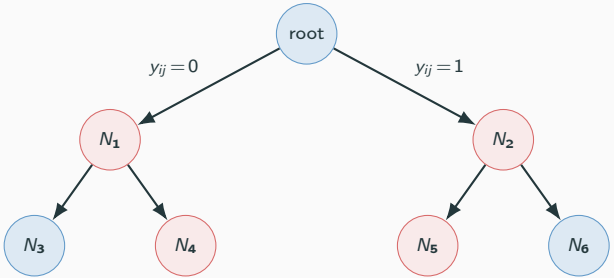
Branch-and-price

1: UB $\leftarrow +\infty$; open $\leftarrow \{\text{root}\}$; global_LB $\leftarrow -\infty$
2: while open non-empty
3: $N \leftarrow$ pick-next(open) # best-first (smallest LB) OR depth-first dive
4: if N.LB $\geq$ UB $- \varepsilon$: prune
5: run CG on N *passing* UB; # CG stops early if $LB \geq UB$
6: global_LB $\leftarrow \min(N.\text{LB}, \min_{M \in \text{open}} M.\text{LB})$
7: if (UB $-$ global_LB)/|UB| $\leq \delta\%$: stop (gap reached)
8: if N.LB $\geq$ UB $- \varepsilon$: prune
9: if LP integer: UB $\leftarrow \min(\text{UB}, z^{\text{RMP}})$; prune
10: else run heuristic (dive at root, RMH every X); update UB
11: pick branching arc (u, v); create down, up children
12: if depth_first: process *up*-child immediately; queue down
13: else: queue both, sort open by LB

Cut-and-price – one sentence

Add valid inequalities (subset-row, capacity, clique…) to the master. Each cut introduces a dual variable that the pricer must include in the reduced cost. Most modern solvers (BaPCod, VRPSolver, GCG, RouteOpt) do this, with the so-called subset-row cuts being particularly potent for VRPTW. Out of scope today – see Desrosiers, Lübbecke, Desaulniers & Gauthier (2026), the open-access book in your folder (ISBN 978-3-031-96917-1), and Pecin et al. (2017) for the canonical SR-cut treatment.

Putting all of B&P on one slide



Each node:
- filter pool by branch
- run CG (with LB)
- prune if LB$\geq$inc.
- run heuristic
- branch

- Solid blue nodes = pruned by bound, integrality (or infeasibility).
- Tree depth $\sim |\mathcal{N}|$ in the worst case but very rarely so in practice.

Branching strategy summary

Rule	Pricing change	Use when
Master x_p (naive)	k-shortest path; bad	never
Ryan-Foster (i,j)	merge / forbid pair	set-partitioning
Arc y_{uv}	remove / contract arc	VRP, network flow
Resource τ_v	split window	deep nodes
Aggregated knapsack	on a_{ip}-vector	cutting stock

Lab choice

We'll implement arc branching – one branch per fractional arc. Easy to apply, easy to filter the column pool by "does the route use arc (u, v)".

Exercise E – B&P with Lagrangian pruning

Exercise E – arc-branching B&P

- Implement `lagrangian_bound(mp_dual_by_id, pricing_sols, K)` – formula on the next slide. The B&P engine uses this to prune.
- Write `aggregated_arc_flow`: $\bar{y}_{uv} = \sum_{p:(u,v)\in p} \bar{x}_p$.
- Pick the most-fractional arc; build the two children (`forbid` / `force`). Implement the pricing-graph edit (`EX-E.5`: skip forbidden arcs).
- Filter the parent column pool: drop columns incompatible with the branch; keep the rest as warm-start.

The B&P loop itself is already coded in `bnp.py`; you fill only the helpers `EX-E.1`, `E.2`, `E.3`, `E.5`. The Lagrangian bound is reused from Exercise C / Trick 7. Knobs to play with at the end: `-gap` (stop within δ%), `-no-depth-first` (pure best-first), `-rmh-every` (heuristic frequency).

Time budget: 35 minutes – end-of-workshop deliverable.

Visualising convergence

Seeing the algorithm

Two helpers ship with the lab kit (`vrp.plot`) and turn the CG / B&P traces into convergence figures:

- `plot_cg_convergence(runs)` – LP & LB per iteration plus the duality gap, for several CG configurations on the same axes. Powered by `CGStats.lp_history`, `CGStats.lb_history` (filled by every CG call).
- `plot_bnp_progress(runs)` – UB / LB and gap per *node processed* (top row) and per *wall-clock second* (bottom row), for several B&P configurations. Powered by the `BnPIncumbent.{ub,lb,time}_history` fields that the B&P engine records as it goes.

Why look at the curves

Picking K, α, gap and search order from a table of final numbers misses the dynamics. The convergence curves show *when* each parameter pays off (early vs late) and reveal tailing-off vs sustained progress.

Comparison runners

```
# CG -- batch sizes
python -m vrp.tests.compare_cg --instance RC101.txt --Ks 1 5 20 50

# CG -- Wentges smoothing
python -m vrp.tests.compare_cg --instance RC101.txt --alphas 0 0.2 0.5

# B&P -- gap thresholds
python -m vrp.tests.compare_bnp --instance R101.txt --gaps 0 2 5

# B&P -- depth-first vs best-first
python -m vrp.tests.compare_bnp --instance R101.txt --search both

# B&P -- pricing batch sizes
python -m vrp.tests.compare_bnp --instance R101.txt --Ks 5 20 50
```

Add `-save figure.png` to write the figure to disk instead of displaying it. Every CG / B&P run inside the comparison uses the same logger, so `-v` also pours iteration logs to stderr.

What the experiments tell you

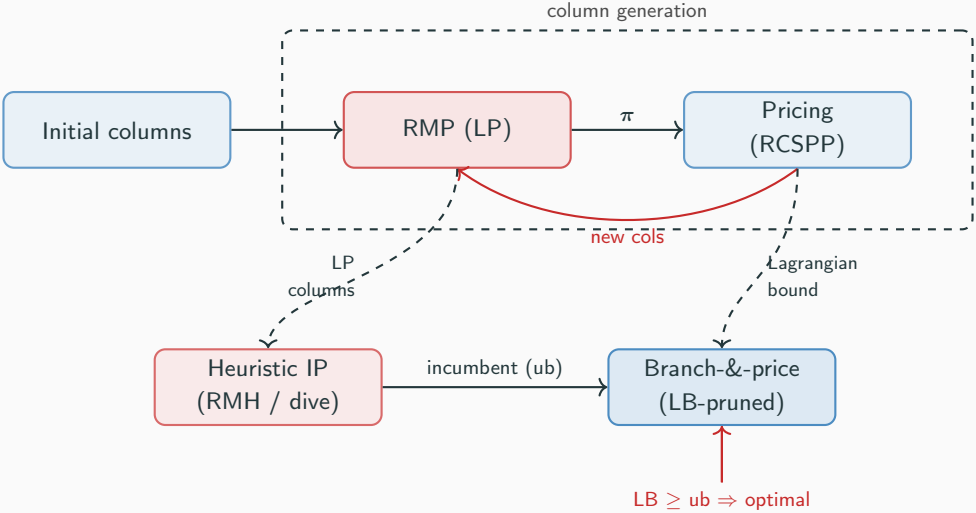
- **K vs iterations** – $K = 1$ takes many iterations but each is cheap; $K = 50$ takes few iterations but each LP is heavier. Expect the wall-clock curve to be U-shaped.
- **Wentges** α – $\alpha \in [0.3, 0.7]$ usually halves iterations on Solomon-100. Too high ($\alpha = 0.9$) makes LB lag, hurting the gap-stop.
- **Gap stop** – on R101, gap=2% typically closes 80% of the optimality gap in 20% of the time.
- **Depth-first vs best-first** – depth-first finds an incumbent faster (good for pruning); best-first closes the global LB faster (good when you want a small certified gap). Hybrid solvers do both.

Recap and references

Recap – what we covered

- **Why**: extended formulations with one variable per combinatorial object beat compact LPs.
- **How**: solve the restricted master, compute duals, price new columns, repeat.
- **Pricer**: RCSPP via label-setting; resources, dominance, ng-routes, multi-column return.
- **Speed**: stabilisation, hierarchy of pricers, warm-start LP, column pool management, tailing-off.
- **Integrality**: RMH, diving, price-and-branch.
- **Optimality**: branch-and-price, smart branching rules, Lagrangian bound for early pruning.

Recap – one diagram



If you remember four things

1. **Don't enumerate**: the master never sees most of $\mathcal{P}$. You generate columns only when their reduced cost is negative.
2. **Branch on structure, not on master variables**: Ryan-Foster, arc branching, resource branching all keep the pricing problem intact.
3. **Use many heuristics** to implement an efficient CG and B&P.
4. **The Lagrangian bound is your friend**: it converts a fractional LP into a true lower bound, kills CG tailing-off, and prunes B&P nodes early.

References (1/3) – foundations

- P. Gilmore & R. Gomory, *A linear programming approach to the cutting stock problem*, Operations Research, 9(6), 849–859, 1961.
- G. Dantzig & P. Wolfe, *Decomposition principle for linear programs*, Operations Research, 8(1), 101–111, 1960.
- L. Lasdon, *Optimization theory for large systems*, Macmillan, 1970 (Lagrangian bound).
- D. Ryan & B. Foster, *An integer programming approach to scheduling*, in Computer Scheduling of Public Transport (A. Wren, ed.), 269–280, North-Holland, 1981.
- M. Desrochers & F. Soumis, *A generalized permanent labelling algorithm for the shortest path problem with time windows*, INFOR, 26(3), 191–212, 1988.
- C. Barnhart, E. Johnson, G. Nemhauser, M. Savelsbergh & P. Vance, *Branch-and-price: column generation for solving huge integer programs*, Operations Research, 46(3), 316–329, 1998.
- J. Desrosiers, F. Soumis & M. Desrochers, *Routing with time windows by column generation*, Networks, 14(4), 545–565, 1984.

References (2/3) – methods & algorithms

- O. du Merle, D. Villeneuve, J. Desrosiers & P. Hansen, *Stabilized column generation*, Discrete Mathematics, 194(1-3), 229–237, 1999.
- P. Wentges, *Weighted Dantzig–Wolfe decomposition for linear mixed-integer programming*, International Transactions in Operational Research, 4(2), 151–162, 1997 (smoothing).
- F. Vanderbeck, *Implementing mixed integer column generation*, in Column Generation (G. Desaulniers, J. Desrosiers & M.M. Solomon, eds.), 331–358, Springer, 2005.
- F. Vanderbeck & L.A. Wolsey, *Reformulation and decomposition of integer programs*, in 50 Years of Integer Programming, 431–502, Springer, 2010.
- R. Baldacci, A. Mingozzi & R. Roberti, *New route relaxation and pricing strategies for the VRPTW*, Operations Research, 59(5), 1269–1283, 2011 (ng-routes).
- D. Pecin, A. Pessoa, M. Poggi & E. Uchoa, *Improved branch-cut-and-price for capacitated vehicle routing*, Mathematical Programming Computation, 9(1), 61–100, 2017 (subset-row cuts).
- N. Boland, J. Dethridge & I. Dumitrescu, *Accelerated label-setting algorithms for the elementary RCSPP*, Operations Research Letters, 34(1), 58–68, 2006 (bidirectional pricing).

References (3/3) – workshop pointers

- J. Desrosiers, M. Lübbecke, G. Desaulniers & J.B. Gauthier, *Branch-and-Price*, Springer, 2026 (open access). ISBN 978-3-031-96917-1, DOI `10.1007/978-3-031-96917-1` (PDF in your workshop folder).
- G. Desaulniers, J. Desrosiers & M.M. Solomon (eds.), *Column Generation*, Springer, 2005.
- M.M. Solomon, *Algorithms for the vehicle routing and scheduling problems with time window constraints*, Operations Research, 35(2), 254–265, 1987 (the benchmark instances we use).
- Open-source LP/MIP: HiGHS (`https://highs.dev`); CBC, GLPK.
- Research B&P solvers: BaPCod, GCG, VRPSolver, RouteOpt.
- Python ecosystem: `rcspp` (workshop), `highspy`, `pulp`, `coptpy`, `networkx`.